NLP Assignment 01

Name : Md Aminul Islam Rony

Roll : ID25M804

Part 1: Sentence Segmentation

Q1) Ans: A simple top down approach for sentence segmentation in English is splitting the text when punctuation mark is encountered at sentence end. For example . ! ? etc.

It can be implemented using rule based method or regular expression which identifies punctuation marks followed by whitespace.

This method can face issues in several situations below with example:

1. **Abbreviations**
   Example: Md. Aminul Islam studies at IIT Madras.
   This naive method may split after Md. which can produce incorrect sentences.
   Strategy to handle issue: By maintaining a list of common abbreviations and avoiding splitting after them.

2. **Decimal Numbers**
   Example: Value of pi is 3.14. It is very important in mathematics.
   Here 3.14 may incorrectly trigger a sentence split.
   Strategy to handle this issue: Using regular expression to detect digits around periods and preventing splitting within numbers.
3. **Quotations**
   Example: He said , “I have exam tomorrow.” The he left.
   Here segmentation may fail due to the punctuation inside quotes.
   Strategy to handle issue: By applying rules that consider quotation marks when detecting sentence boundaries.
4. **Ellipsis**
   Example: Wait….. I think we shall wait for a while.
   Here Naive segmentation might incorrectly split the sentence after each period.

Strategy to handle issue: By Detecting sequences such as "..." and treat them as a single punctuation unit.

Difference of NLTK Punkt Sentence segmenter from a Simple top down approach:
The Punkt sentence segmenter in NLTK uses an unsupervised statistical approach rather than simple punctuation rules. It learns patterns such as abbreviations, sentence starters and punctuation usage from a training corpus. For this reason, it can distinguish between periods used in abbreviations (e.g. Md., *Dr.*, *U.S.*) and actual sentence boundaries.

A simple rule based method splits the sentences using punctuation marks like ., !, or ?. It does not consider context, so it often incorrectly splits sentences containing abbreviations, decimals or titles.
Example:
Md. Aminul lives in Chennai.
A naive rule based method may incorrectly split after **Md.** In the other hand, Punkt correctly identifies it as an abbreviation and keeps the sentence intact.

Comparison with spaCy Sentence Segmentation:

Method	Approach	Characteristics
Naive rule based	It used top down rules.	It is simple and fast but fails with abbreviations and special cases.
NLTK Punkt	It is statistical / unsupervised	It learns abbreviation and boundary patterns from data.
Spacy	It uses hybrid rules + linguistic features.	It is more robust and accurate for complex text structures.

Q2) Ans:

a) 15 sentences :
   1. Md. Aminul studies at IIT Madras.
   2. Mr. Rony had his dinner at 8.30 p.m. yesterday.

3. Pi is equal to 3.14. It is very important in mathematics.
4. He lives in U.K and works at a burger shop.
5. Prof. Sudarsun wrote the email.
6. Wait….. I think we shall wait for a while.
7. He said, “It stopped raining.” Then we left for the class.
8. The meeting starts at 11 a.m. Please be punctual.
9. Are you serious?! I can’t believe it.
10. The temperature dropped from 20.5°C to 18.2°C overnight.
11. I bought organs, apples etc. from the shop.
12. This system-version 8.2- is faster than the before.
13. The website is www.aminulrony.com. It is a portfolio website.
14. I met Mr. and Mrs. Bose yesterday.
15. I stopped…then continued walking.

b) Categorization by Error Type:

Sentence	Error Type
1.Md. Aminul studies at IIT Madras.	Abbreviation
2. Mr. Rony had his dinner at 8.30 p.m. yesterday.	Abbreviation + time notation
3) Pi is equal to 3.14. It is very important in mathematics.	Decimal numbers
4) He lives in U.K and works at a burger shop.	Abbreviation
5) Prof. Sudarsun wrote the email.	Abbreviation
6) Wait….. I think we shall wait for a while.	Ellipsis
7) He said, “It stopped raining.” Then we left for the class.	Quotes
8) The meeting starts at 11 a.m. Please be punctual.	Time abbreviation
9) Are you serious?! I can’t believe it.	Multiple punctuation
10) The temperature dropped from 20.5°C to 18.2°C overnight.	Decimal numbers
11) I bought organs, apples etc. from the shop.	Abbreviation
12) This system-version 8.2- is faster than the before.	Decimal number

13) The website is www.aminulrony.com. It is a portfolio website.	URL
14) I met Mr. and Mrs. Bose yesterday.	Abbreviations
15) I stopped…then continued walking.	Ellipsis

(c) Error Evaluation of Segmenters:

```
...
>>> print("Naive errors:", naive_errors)
Naive errors: 15
>>> print("Punkt errors:", punkt_errors)
Punkt errors: 4
>>> print("spaCy errors:", spacy_errors)
spaCy errors: 4
```

I found by running a program:
Naive Errors : 15
Punkt Errors: 4
SpaCy Errorrs : 4

Part 02: Tokenization

Q1) Ans: In my opinion, a simple top down approach for tokenization in English is splitting text based on whitespace and punctuation marks. In this case, each word becomes separated by spaces. It is treated as a token. In the other hand, punctuation symbols are removed using simple rules or regular expression.

For example : “My name is Aminul “

Tokens of the above sentence : [“My”,”name”,”is”,”Aminul”]

This approach is simple and computationally efficient. But it lacks of linguistic awareness.

Situations where the approach may fail :

1. **Contractions :**
   He isn't an honest man.
   Token : He, is, nt, an, honest, man
2. **Hyphenated words:**

   This is a state-of-the-art system.

   Tokens: This, is, a, state, of, the , art, system

3. **Decimal numbers:**
   Pi is equal to 3.14
   Tokens : Pi, is , equal, to, 3, 14
4. **URL:**
   www.aminulrony.com
   token: www, aminulrony, com
5. **Email**
   24im0022@iitism.ac.in
   Token: 24im0022, @, iitism, ac,in

Q2) Ans:

NLTK's Penn treebank tokenizer uses top down approach. It works based on a set of predefined linguistics rules. These rules determine how punctuation, contractions and special characters will be separated from words. As the rules are applied directly to the text without learning from data, it is considered to be a top-down approach.

In the other hand, the spaCy tokenizer combines rule based tokenization as well as linguistic knowledge and exception handling. It uses prefix, suffix and infix rules along with special case dictionaries

to correctly identify tokens in different contexts. It combines several techniques rather than relying solely on rules or data driven learning. So, it is a hybrid approach.

Q3) Ans:

I have performed word tokenization of the sentence segmented documents using top-down approach, Penn Treebank Tokenizer and spacCy tokenizer in my programming part.

<u>Scenario Where the Proposed Approach Performs Better :</u>

The naive top-down tokenizer performs better where punctuation noise needs to be removed completely. For instance, texts which contain repeated punctuation or informal writing.

Example: Hurray!!! We have won the game.

Naive tokenizer output : “Hurray”, “We”,”have”,”won”,”the”,”game”

PennTreebank
output:“Hurray”,”!”,”!”,“!”,“We”,”have”,”won”,”the”,”game”

spacCy output :
“Hurray”,”!”,”!”,”!”,”We”,”have”,”won”,”the”,”game”

The naive top down tokenizer removes all punctuation and produces cleaner token.

<u>Scenario Where the Proposed Approach Performs Worse:</u>

The naive top down tokenizer performs worse while dealing with complex linguistics pattern and contractions.

Example : I won’t go to class today.

Naive output : “I”,”won”,”t”,”go”,”to”,”class”,”today”

Penn Treebank output: “I”,”wo”,”n’t”,”go”,”to”,”class”,”today”

spaCy output: “I”,”wo”,”n’t”,”go”,”to”,”class”,”today”

The naive top down tokenizer incorrectly splits the contraction “won’t” into won and t. But the other two tokenizer correctly identify the contraction and split it into “wo” and “n’t” which preserves linguistic structure of the sentence.

Part 3: Stemming and Lemmatization

Q1)Ans:

Stemming : It reduces word to its root form by removing suffixed using heuristic rules and doesn’t consider the word’s meaning or grammatical context.

For example : Running (word) -> run (stem)

Studies (word) -> studi (stem)

Computing (word) -> comput

Lemmatization: It reduces to word to its dictionary base form name lemma using linguistic knowledge such vocabulary and morphological analysis.

Example:

Running -> run (lemma)

Studies -> study (lemma)

Better -> good (lemma)

Conceptual Differences:

Aspect	Stemming	Lemmatization
Linguistic Knowledge	It uses simple rules without linguistic knowledge	It uses dictionaries and morphological analysis.
Output form	It may produce non dictionary words.	It produces valid dictionary word.
Computational complexity	Low and faster	Higer and more expensive

Q2)Ans:

I have performed stemming using Porter's stemmer and lemmatization using WordNet Lemmatizer using the word tokenized text from Cranfield Dataset.

Output of the tokenization :

ORIGINAL TOKENS	PORTER STEMMER	WORDNET LEMMATIZER
engines	engin	engine
the	the	the
matrices	matric	matrix
better	better	better
organization	organ	organization
testing	test	testing
for	for	for
universal	univers	universal
are	are	are
aerodynamic	aerodynam	aerodynamic
ran	ran	ran
these	these	these
universities	univers	university
of	of	of
the	the	the
in	in	in
caring	care	caring

Comparision of the outputs of the two methods by analyzing:

Changes in vocabulary size:
Stemming:
Stemming reduces the vocabulary size because it removes suffixes without considering context or dictionary lookup. As a result, several different words are mapped into the same morphological root, grouping many variations into a single form.

Lemmatization:
Lemmatization usually results in a comparatively larger vocabulary. This happens because it relies on dictionary-based root forms and preserves distinctions between different words instead of merging them aggressively.

Examples of Over-Stemming:

Some words lose their proper meaning when stemming is applied:
aerodynamic → aerodynam
universal and universities → univers
organization → organ
matrices → matric

By these examples, we can see how stemming can truncate words excessively and produce forms that are not valid words.

Cases Where Lemmatization Preserves Semantic Meaning Better:
Lemmatization performs better when handling irregular word forms and maintaining correct meanings.

It correctly converts the irregular plural **matrices → matrix**.

It also transforms **universities → university**, preserving the proper base form.

Part 4: Stopword Removal

Q1)Ans:
I have implemented it in the coding part.

Q2) Ans:
By calculating the frequency of distinct word in the corpus and sorting them in descending order. Those words will have frequency exceeding a chosen threshold (top k frequent words), we can label those words as stopwords. My intuition behind that , highly frequent words have low discriminative power. As those words appear in many documents and do not help us to distinguish relevant content.

Q3)Ans:
The above mentioned strategy was implemented in coding part.
Here, I will show the comparision.

Category	Explanation
NLTK Stopword List	Contains about 179 predefined English stopwords.
Data-Driven Stopwords	Selects the top K most frequent words in the corpus based on a chosen hyperparameter K.
Overlap	High-frequency structural words such as "the" and "this" appear in both lists.

Appears in NLTK but not in Custom	Words like "of", "is", and "for" appear in NLTK but not in the custom list because K = 5 was small.
Appears in Custom but not in NLTK	Domain-specific words such as "aerodynamics", "paper", and "method" appear frequently in the corpus and are identified as stopwords.

Part 5: Retrieval

Q1)Ans:
A similarity score must be computed between **each query and each document**. Therefore, the total number of similarity computations is:

$$|Q| \times |D|$$

Each query and document is represented as a TF-IDF vector of size $|V|$, so computing one similarity (dot product) requires **O(|V|)** operations. For that reason, the total computational cost is below: **O(|Q| × |D| × |V|)**.

Q2)Ans:

In an inverted index, each term in the vocabulary is mapped to the list of documents that contain that term. Instead of comparing every query with every document in the corpus, we only retrieve the documents that contain at least one of the query terms.

Additional variables:

- **Lavg:** The average number of terms in a query.
- **s:** The sparsity factor of the corpus, defined as the fraction of documents containing a specific term ($s << 1$).
- **Pavg:** The average length of a posting list in the inverted index. It is calculated as:

$$\mathbf{Pavg = s \times |D|}$$

Naive Approach:

In the naive approach, every query is compared with every document and each similarity computation uses **TF-IDF** vectors of size **|V|.**

Complexity: $\mathbf{O(|Q| \times |D| \times |V|)}$

Inverted Index Approach:

Using an inverted index, only the documents that contain the query terms are considered. For each query, only Lavg terms are processed, and each term retrieves about Pavg documents from the posting list.

Complexity: $\mathbf{O(|Q| \times Lavg \times Pavg)}$

Comparison:

Since **Lavg << |V|** and **Pavg << |D|** due to the sparsity of the corpus, the inverted-index-based approach significantly reduces the number of computations compared to the naive method.